

MEDIAN

GDD COMPILER

Python Process and Implementation Specification

A reusable, phase-gated workflow for compiling multi-source design revisions

Version 1.0
31 July 2026
Implementation handoff

Executive Summary

This document specifies a small, reusable Python program for supervising the MEDIAN GDD compilation process. The program is deliberately conservative: it performs deterministic file handling, normalization, validation, caching, and artifact assembly; delegates bounded structured extraction to Claude; delegates judgment-heavy comparison and audit tasks to a configurable OpenAI reasoning model; and reserves all canon authority for the human operator.

The implementation is developed and operated in phases. Each transformation is first perfected on representative source documents, frozen with a versioned ruleset and acceptance tests, then applied corpus-wide. Later phases are not permitted to repair preventable damage introduced earlier.

Central rule: The program manages the process. It does not become the canon authority, and no generated summary replaces the complete source passages.

Concern	Practical decision
Program form	A local Python command-line application inside the existing MEDIAN Git repository.
Primary data	Markdown for source text; JSONL for atomic records and rulings; CSV for compact manifests and reports; YAML for configuration.
Extraction model	Claude, used for source-local atomic decomposition and classification.
Judgment model	A configurable OpenAI reasoning model, used for bounded conflict analysis, terminology reconciliation, architecture support, and audits.
Human role	Final authority for canon rulings, terminology, target architecture, deferrals, and release acceptance.
Development method	Perfect one phase, freeze it, run it across the complete corpus, commit, then begin the next phase.
Initial scope	Corpus manifest, faithful normalization, lean Markdown, deterministic chunking, Claude extraction, extraction QA, and conflict preparation.

Part I — Program Specification

1. Purpose and Design Position

The compiler converts one canonical baseline GDD, multiple update specifications, decision and audit passes, later human rulings, and associated figures or tables into a controlled build corpus suitable for reconciliation and eventual section-by-section compilation. It is intended to be reused for v0.6 and later editions, with version-specific behavior supplied by sources, manifests, prompts, schemas, and rulings rather than hard-coded into the program.

- **The ledger determines what happens.** Structured records and dispositions control the build.
- **The sources determine what is true.** Full operative passages remain available and traceable.
- **The target architecture determines where it belongs.** Document order is decided separately from source-processing order.
- **The final GDD states only the resulting world.** Design debate and version archaeology are archived rather than leaked into published prose.

1.1 Non-goals

- No autonomous canon rulings.
- No whole-document “agent” with unrestricted repository access.
- No requirement for a database, web interface, vector store, or Google Sheets integration in the first implementation.
- No silent overwriting of accepted human rulings.
- No dependence on one vendor-specific model name; provider and model IDs remain configuration values.
- No attempt to solve target GDD order during source extraction.

2. Minimal Technical Architecture

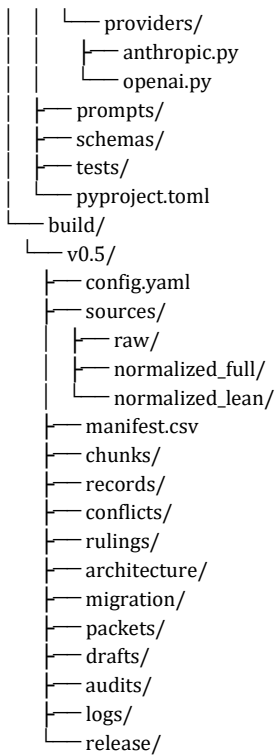
The first implementation should be intentionally boring. A command-line interface coordinates files and API calls. Every stage writes inspectable artifacts to disk. Git records phase gates and accepted changes.

Layer	Responsibility	Recommended technology
Deterministic utility layer	Hashing, file discovery, conversion, boilerplate rules, chunking, validation, report generation, caching, packet assembly.	Python 3.12+, Typer, Pydantic, Rich, standard library.
Extraction provider	Atomic source decomposition and source-local consolidation.	Anthropic Python SDK; configured Claude model.
Judgment provider	Conflict classification, terminology reconciliation, architecture support, semantic audits.	OpenAI Python SDK; configured reasoning model.
Human gate	Canon decisions, terminology approval, architecture approval, release acceptance.	Terminal review packets and version-controlled ruling records.
Artifact store	Immutable sources, normalized text, JSONL records, CSV reports, Markdown decisions and drafts.	Repository filesystem and Git.

```

median/
├── compiler/
│   ├── median_compile/
│   │   ├── cli.py
│   │   ├── config.py
│   │   ├── models.py
│   │   ├── manifest.py
│   │   ├── normalize.py
│   │   ├── lean.py
│   │   ├── chunk.py
│   │   ├── extract.py
│   │   ├── judge.py
│   │   ├── reconcile.py
│   │   ├── packets.py
│   │   └── audit.py

```



3. Data Formats

Use the least elaborate format that preserves the structure of each artifact. Human readability, Git diffs, and token economy take priority over database sophistication.

Artifact	Format	Reason
Source manifest	CSV	Compact, diffable, easy to inspect and validate.
Full and lean normalized sources	Markdown	Preserves headings, tables, formulas, and source coordinates with low token overhead.
Atomic canon records	JSONL	One independently valid record per line; supports arrays and long passages without CSV contortions.
Conflict candidates and judgments	JSONL	Structured, appendable, and suitable for bounded API packets.
Human rulings	JSONL plus Markdown report	Machine-readable authority record with a readable decision log.
Configuration	YAML	Human-editable provider, model, path, and gate settings.
Coverage and status reports	CSV and Markdown	CSV for metrics and filtering; Markdown for review.
Target architecture and Change Guide	Markdown	Long-form human-approved editorial artifacts.

3.1 Compact atomic record

```

{"id":"SPEC_HOME:0042","src":"SPEC_HOME","loc":"§4.2¶3","quote":"Each present and available Citizen contributes one unit of Capacity..." ,"claim":"Each available Citizen contributes one equal unit of Home Role Capacity","type":"rule","weight":"STATE","status":"canonical","owner":"home.roles","terms":["Citizen","Capacity"],"deps":[],"flags":{}}
  
```

Short keys are acceptable in storage because schemas and reports expand them for humans. The full operative quotation remains attached during extraction and audit; later comparison packets may include only the records relevant to the judgment.

4. Model Allocation and Authority

4.1 Claude: structured extraction

- Segment lean source chunks into atomic claims.
- Preserve operative source quotations and coordinates.
- Classify content type, publication weight, normative status, terminology, explicit dependencies, and explicit supersession.
- Identify examples, guardrails, tuning notes, visual requirements, and open questions.
- Perform source-local consolidation of overlap duplicates and split claims.
- Flag possible collisions without resolving canon.

4.2 OpenAI reasoning model: judgment-heavy work

- Determine whether record clusters are duplicate, compatible, scoped refinements, terminology conflicts, or direct contradictions.
- Analyze system ownership and source precedence without converting recommendations into rulings.
- Reconcile terminology candidates and false synonyms.
- Audit ecology against Core Species and Crossing material.
- Support concept-dependency and target-architecture analysis.
- Audit section packets and compiled sections for omissions, weakened conditions, unsupported additions, and terminology drift.

4.3 Human canon authority

- Approve or reject canon rulings.
- Promote provisional or research material into canon.
- Choose canonical terminology where sources disagree.
- Approve target GDD architecture and appendices.
- Decide deferral versus release blocking.
- Accept the release candidate.

Provider rule: Model names are configuration values. The implementation must not depend on an informal product nickname or on the current availability of a specific model.

5. Phase-Gated Development and Operation

The compiler is both built and operated in phases. Each phase has a pilot, a validation gate, a versioned ruleset or schema, a corpus-wide run, and a Git commit or tag. The next phase begins only after the preceding phase has passed.

Phase	Purpose	API use	Gate output
0. Corpus and manifest	Freeze sources, assign IDs, hash files, classify source status.	None	Validated manifest and immutable raw corpus.
1. Full Markdown normalization	Produce faithful, coordinate-preserving Markdown.	None	Validated full Markdown for every source.
2. Lean Markdown	Remove deterministic boilerplate without losing possible canon.	None	Frozen lean ruleset, removal ledger, token report.
3. Structural segmentation	Create stable, bounded chunks that preserve semantic units.	None	Validated chunk registry.
4. Claude extraction	Produce atomic records and source-local consolidation.	Claude	Validated record schema and prompt, then corpus-wide records.
5. Extraction QA	Prove quotation, coordinate, paragraph, table, and formula coverage.	Optional bounded audit	Coverage reports with no unexplained omissions.
6. Lexicon candidates	Extract terms, aliases, definitions, and conflicts.	Claude or OpenAI	Candidate terminology registry.
7. Conflict candidate generation	Use deterministic matching to build small comparison clusters.	None	Known conflicts recalled; manageable candidate volume.

Phase	Purpose	API use	Gate output
8. Judgment	Classify bounded clusters and recommend resolution paths.	OpenAI	Structured judgments, never automatic rulings.
9. Human rulings	Resolve or explicitly defer conflicts.	None	Immutable human ruling records.
10. Target architecture	Design and freeze the v0.5 GDD order and section contracts.	OpenAI support, human approval	Frozen section IDs, purposes, dependencies, appendices.
11+. Migration and compilation	Atomize baseline, build Migration Ledger, Change Guide, packets, drafts, audits, release.	Both, in bounded tasks	Verified compiled edition and supporting artifacts.

5.1 Phase 0 — Corpus and manifest

Create the minimal source index and verify that the build is operating on the intended corpus.

- Copy or reference immutable raw files.
- Assign stable source IDs independent of filenames.
- Record version, source class, maturity, scope, declared supersession, dependencies, inclusion status, and SHA-256 hash.
- Reject duplicate IDs, missing files, invalid dependencies, and superseded sources incorrectly marked active.

5.2 Phase 1 — Full Markdown normalization

Perfect faithful conversion before attempting token reduction.

- Preserve heading hierarchy, lists, tables, formulas, callouts, captions, footnotes, and page/source coordinates.
- Create a Figure Registry for diagrams and image-based tables.
- Validate several difficult document families before running corpus-wide.
- Freeze the normalizer version and rerun only when source hashes or conversion rules change.

5.3 Phase 2 — Lean Markdown

Create the token-efficient working source used for serious LLM operations.

- Remove only recognized boilerplate: repeated headers and footers, page furniture, decorative separators, duplicate title blocks, duplicate contents material, and metadata already captured in the manifest.
- Retain definitions, rules, exceptions, examples, tables, formulas, explicit scope, precedence, supersession, open questions, and mechanical non-goals.
- Log every removed block with the rule, line range, and hash.
- Produce full-versus-lean token estimates and reduction percentages.
- Freeze a versioned lean ruleset only after manual comparison on representative documents.

5.4 Phase 3 — Structural segmentation

Create stable chunks without asking an LLM to repair arbitrary text breaks.

- Split by heading, then paragraph boundaries, under a configurable token ceiling.
- Keep tables, procedures, formulas with definitions, and qualifying callouts intact.
- Assign stable chunk IDs and content hashes.
- Validate that every paragraph occurs in at least one chunk and overlap remains controlled.

5.5 Phase 4 — Claude extraction

Use bounded, schema-constrained calls to produce atomic records.

- Pass A extracts candidate records from each chunk.
- Pass B consolidates source-local overlap and repairs split claims.
- A coverage ledger assigns each lean paragraph to record IDs or to an explicit non-record classification.
- Pilot and revise until source quotations, conditions, exceptions, tables, and open questions are reliably preserved.
- Freeze the prompt and schema before corpus-wide extraction.

5.6 Phase 5 — Extraction QA

Validate extraction before any cross-source comparison.

- Check record IDs, controlled values, quotation existence, coordinate validity, paragraph coverage, formula coverage, and table coverage.
- Flag claims stronger than their quotations and examples misclassified as rules.
- Use fresh-context semantic audits only on bounded packets.
- No cross-source judgment begins while a source is known to be incompletely atomized.

6. Lean Markdown Process

Lean Markdown is the most important early engineering phase because every later LLM operation inherits both its token savings and its omissions. The leaner must be deterministic, conservative, reversible, and independently testable.

6.1 Dual outputs

```
sources/normalized_full/SPEC_HOME.md
sources/normalized_lean/SPEC_HOME.md
reports/lean/SPEC_HOME.removals.jsonl
reports/lean/corpus_summary.csv
```

6.2 Safe first-pass removals

- Repeated running headers and footers.
- Standalone page numbers.
- Decorative separators and blank decorative pages.
- Repeated document title, version, and target labels.
- Duplicate contents pages once heading structure has been validated.
- Exact duplicate metadata blocks already captured in the manifest.

6.3 Material not automatically removed

- Precedence statements.
- Scope boundaries and exclusions.
- Explicit supersession language.
- Definitions and terminology notes.
- Revision mandates containing operative rules.
- Non-goals with mechanical consequences.
- Examples, tables, formulas, captions, and figure references.
- Open questions and deferred items.

6.4 Lean acceptance report

Metric	Requirement
Character and token counts	Report full, lean, and percentage reduction.
Removal count	Count removals by rule and document.
Recoverability	Every removal has line range and hash in the full normalized source.
Continuity	Lean Markdown remains readable and heading-consistent.
Canon preservation	Manual review finds no lost rule, definition, exception, example, formula, table, or unresolved item.
Versioning	Every file records the lean ruleset version and source hash.

7. Idempotence, Caching, and Invalidation

Every command should be safe to rerun. Model calls are keyed by normalized input hash, prompt version, schema version, provider, model ID, and relevant configuration. Unchanged work is reused rather than purchased again.

- Changed raw-source hash invalidates normalization and all downstream artifacts for that source.
- Changed lean ruleset invalidates lean files, chunks, extraction, and downstream records.
- Changed chunker version invalidates chunk-scoped extraction but not normalized sources.

- Changed extraction prompt or schema invalidates extraction outputs.
- Changed judgment prompt invalidates judgments but never accepted human rulings.
- Changed ruling marks affected downstream migration and section packets stale.

```
{
  "call_id": "call_00481",
  "provider": "anthropic",
  "model": "${ANTHROPIC_EXTRACTION_MODEL}",
  "task": "extract",
  "source": "SPEC_HOME",
  "chunk": "C004",
  "input_hash": "...",
  "prompt_version": "extract-1.0",
  "schema_version": "records-1.0",
  "input_tokens": 11842,
  "output_tokens": 4630,
  "status": "ok"
}
```

8. Command-Line Interface

Commands should expose explicit stages rather than a single “run everything” action. A convenience command may later chain already-approved phases, but each phase must remain independently inspectable and rerunnable.

```
median-compile init build/v0.5
median-compile manifest build/v0.5
median-compile normalize-full build/v0.5
median-compile normalize-lean build/v0.5
median-compile report-lean build/v0.5
median-compile validate-normalization build/v0.5
median-compile chunk build/v0.5
median-compile extract build/v0.5 --source SPEC_HOME
median-compile validate-extraction build/v0.5 --source SPEC_HOME
median-compile extract-all build/v0.5
median-compile build-lexicon build/v0.5
median-compile detect-conflicts build/v0.5
median-compile judge-conflicts build/v0.5
median-compile review-conflicts build/v0.5
median-compile architecture build/v0.5
median-compile atomize-baseline build/v0.5
median-compile migration build/v0.5
median-compile change-guide build/v0.5
median-compile packet build/v0.5 --section GDD-07.03
median-compile compile-section build/v0.5 --section GDD-07.03
median-compile audit-section build/v0.5 --section GDD-07.03
median-compile status build/v0.5
```

9. Configuration

```
edition: "0.5"

providers:
  extraction:
    provider: anthropic
    model: "${ANTHROPIC_EXTRACTION_MODEL}"
    max_output_tokens: 12000
  judgment:
    provider: openai
    model: "${OPENAI_JUDGMENT_MODEL}"
    reasoning_effort: high

chunking:
  target_tokens: 10000
  max_tokens: 14000
```



```
overlap_tokens: 400
respect_headings: true
```

```
human_gates:
  conflict_rulings: true
  terminology: true
  architecture: true
  release: true
```

```
versions:
  normalizer_full: "1.0"
  lean_ruleset: "1.0"
  chunker: "1.0"
  extraction_prompt: "1.0"
  record_schema: "1.0"
  judgment_prompt: "1.0"
```

10. Prompt and Packet Discipline

Prompt size is controlled by preprocessing and record reuse, not by asking models to remember the whole project. Each call receives only the material needed for one bounded task.

- Extraction calls receive source ID, coordinate range, compact task contract, controlled vocabularies, JSON schema, and one source chunk.
- Consolidation calls receive candidate records and minimal heading or overlap context, not the full document.
- Conflict calls receive only the relevant records, full quotations, source metadata, and the approved precedence policy.
- Section audits receive one Section Contract, one complete Section Packet, one draft, and one rubric.
- Full quotations are sent only when a judgment must be grounded in them; later indexing uses record IDs and normalized claims.

11. Human Review and Ruling Capture

The first implementation needs no graphical interface. It should generate compact Markdown review packets and offer a terminal choice or accept a manually edited ruling file.

CONFLICT C-0017 — Tool Strain

A. PASS_OVER:0188
Tools do not undergo routine wear.

B. SPEC_ITEMS:0094
After a successful Tool-supported Approach, check Strain.

Model assessment: DIRECT CONFLICT
Recommendation: dedicated equipment specification likely controls.

Choose:
[1] A controls
[2] B controls
[3] Compatible after scope clarification
[4] Defer
[5] Write custom ruling

```
{"id":"RULING-2026-08-01-001","subject":"Tool
Strain","decision":"...","authority":"human","affects":["PASS_OVER:0188","SPEC_ITEMS:0094"]}
```

12. Implementation Milestones

Milestone	Deliverable	Completion condition
A. Corpus and normalization	Manifest, full Markdown, lean Markdown, removal ledger, token report, validation report.	All current sources pass; tag compiler-normalization-v1.

Milestone	Deliverable	Completion condition
B. Chunking	Stable chunks and registry.	No paragraph or table loss; stable IDs; tag compiler-chunking-v1.
C. Claude extraction pilot	Record schema, prompts, Home Loop extraction, coverage report.	Manual audit demonstrates acceptable atomization and traceability.
D. Corpus extraction	All source records and source-local consolidation.	Every source passes extraction QA.
E. Judgment pipeline	Candidate matcher, OpenAI judgment packets, human ruling capture.	Known conflicts recalled and reviewed.
F. Architecture and migration	Target architecture support, baseline records, Migration Ledger, Change Guide.	Approved GDD order and complete bidirectional accounting.
G. Section compilation	Packets, drafts, audits, integration, release.	All acceptance tests and independent audits pass.

13. Minimum Test Suite

- Manifest: unique IDs, existing paths, valid controlled values, valid dependency and supersession references.
- Normalizer: page and heading accounting, table retention, formula retention, figure references, character-loss thresholds.
- Leaner: removal rule snapshots, no protected-pattern removals, reversible removal hashes, token report consistency.
- Chunker: coverage, no split tables or procedures, stable IDs, overlap bounds.
- Extraction: Pydantic validation, quotation substring or coordinate match, controlled values, record-ID uniqueness.
- Coverage: every substantive paragraph represented or explicitly classified.
- Conflict matching: recall tests for known MEDIAN collisions and terminology changes.
- Rulings: immutable accepted decisions and correct downstream invalidation.

14. Immediate Build Handoff

The first developer should implement only Milestone A. No API integration is required until full and lean normalization have been perfected and applied to the complete current corpus.

1. Create the package, CLI shell, configuration loader, source manifest model, and build-directory initializer.
2. Implement source hashing and manifest validation.
3. Implement DOCX-to-full-Markdown conversion with source coordinates and figure inventory hooks.
4. Implement deterministic lean rules with a reversible removal ledger.
5. Implement token estimates, corpus summary, and normalization validation reports.
6. Pilot on Home Loop, Personal Items, Ecological Influences, Overarching Systems, and one short recent document.
7. Freeze and tag the normalization pipeline only after manual comparison passes.

Do not overbuild: No provider SDK, conflict engine, GUI, database, vector search, or section compiler belongs in the first milestone.

Appendix A — Full Underlying Compilation Protocol

The following protocol is the governing process that the Python supervisor is intended to implement incrementally. It is included here so the implementation document remains intelligible without a separate reference file. The Python program may automate, validate, and package this process; it may not collapse its distinct stages or assume authority reserved for human canon decisions.

A1. Purpose

This procedure converts one existing canonical GDD, multiple detailed update specifications, accompanying decision logs, diagrams, tables, formulas, examples, design callouts, and explicit human rulings into one coherent new GDD edition without losing mechanics, guardrails, voice, inherited material, or provenance.

The new GDD is a compiled edition, not a summary and not an ordinary rewrite.

Governing principle: The ledger determines what happens. The sources determine what is true. The target architecture determines where it belongs. The final GDD states only the resulting world.

The process separates source extraction, canon identification, conflict resolution, document architecture, prose composition, and verification. No pass may silently perform the work of another.

A2. Core Integrity Rules

Immutable-source rule: original sources remain unchanged and traceable.

No-summary-substitution rule: no Change Guide, synopsis, migration table, or model summary becomes sole authority.

Atomic-accounting rule: every meaningful source claim receives an explicit disposition.

Positive-canon rule: the final GDD states what MEDIAN is, not the debate that produced it.

No-invented-resolution rule: unresolved design questions require explicit rulings.

Architecture-before-prose rule: target structure is approved before section drafting.

Bounded-compilation rule: sections are written from complete mechanically assembled packets.

Independent-audit rule: a fresh context verifies each completed section.

A3. Artifact Set

Build directories cover raw and normalized sources, manifest, canon registry, conflicts and decisions, target architecture, migration ledger, Change Guide, section packets, drafts, integrated drafts, QA, release, and archive.

Release artifacts include the GDD, Source Manifest, Canonical Requirements, Lexicon, Target Architecture, Migration Ledger, Conflict and Decision Ledger, Change Guide, Internal Tuning Notes, Coverage Report, and Build Record.

Git or equivalent version control is used, with commits after each major gate.

A4. Roles and Model Allocation

Deterministic utilities perform inventory, hashing, conversion, heading and table detection, formula and figure inventory, concatenation, ID assignment, grep tests, cross-reference validation, and coverage calculations.

A judgment model performs requirement decomposition, publication-weight assignment, collision detection, architecture support, terminology reconciliation, conflict analysis, and semantic audits.

A compilation model handles dense section prose only after packets are complete and unambiguous.

The human decides unresolved contradictions, intentional changes, chapter architecture, canonical terminology, deferrals, and final acceptance.

A5. Stage 0 — Freeze and Inventory

Copy all original sources and assets into the raw corpus.

Assign stable source identifiers independent of filenames.

Create the Source Manifest with title, version, date, scope, authority type, supersession, dependencies, hash, page count, extraction status, and notes.

Freeze a provisional precedence policy: explicit human rulings first; explicit supersession over implied; normative rule over example; domain owner over incidental reference; compatible specific refinement over general rule; date alone does not determine authority; UI does not invent mechanics; SAY does not override STATE; unresolved conflicts remain unresolved.

Gate 0: no analysis begins until every source is preserved and identified.

A6. Stage 1 — Normalize Sources

Convert each document separately into validated Markdown.

Preserve headings, lists, tables, formulas, blockquotes, captions, footnotes, callouts, page boundaries, and figure references.

Retain source coordinates in the normalized text.

Create a Figure Registry recording origin, page, caption, type, semantic content, required disposition, target, and validation status.

Validate extraction against originals before concatenating a retrieval master.

Gate 1: every source has a validated normalized copy, coordinates, and figure inventory.

A7. Stage 2 — Atomic Canon Records

Segment every independently actionable requirement, distinction, prohibition, example, tuning assumption, visual instruction, or editorial instruction.

Assign typed record IDs such as REQ, GUARD, TERM, UI, EXAMPLE, SAY, TUNE, OPEN, and HISTORY.

Separate publication weight—STATE, SHOW, SAY, SILENT—from content type.

Each record preserves source, location, verbatim passage, normalized claim, content type, publication weight, normative status, owner, dependencies, conflicts, baseline relation, target, carrier, and notes.

Apply the lantern test: world-facing statements differ from design-process commentary.

Detect silent reversals and pure rejections without manufacturing positive canon.

Gate 2: every meaningful passage has records, status, and publication weight.

A8. Stage 3 — Canonical Terminology Registry

Extract defined terms, capitalized system terms, variables, UI labels, statuses, roles, place names, synonyms, and inconsistent spellings.

Record canonical form, definition, capitalization, singular and plural, aliases, forbidden uses, first-definition location, related terms, audience, and source authority.

Resolve false synonyms and separate internal vocabulary from published vocabulary.

Gate 3: drafting cannot begin while materially important terms retain unresolved definitions.

A9. Stage 4 — Conflicts and Canon Decisions

Detect direct and indirect contradictions in values, formulas, triggers, ownership, timing, costs, outcomes, exceptions, terminology, UI visibility, examples, and double charges.

Create structured Conflict Records with involved passages, conflict nature, precedence analysis, options, recommendation, ruling, authority, date, and affected records.

Use explicit states: resolved by source precedence, resolved by human ruling, compatible after scope clarification, intentionally provisional, deferred, or unresolved blocker.

Convert each human ruling into a source record with provenance.

Audit examples and mockups that accidentally settle supposedly deferred questions.

Gate 4: no section with a blocking unresolved conflict proceeds.

A10. Stage 5 — Target Architecture

Write an editorial contract defining reader, GDD/sourcebook balance, appendix policy, repetition, and technical versus atmospheric prose.

Write a voice constitution covering paragraph length, headings, capitalization, tense, examples, callouts, tables, and in-world language.

Build a concept dependency graph and draft a new table of contents based on reader needs rather than design chronology.

For each section define purpose, reader question, definitions, inclusions, exclusions, dependencies, SHOW and SAY elements, and source clusters.

Map inherited chapters into the new structure, approve architecture independently of prose, and freeze headings, numbering, figures, tables, and cross-reference rules.

Gate 5: target architecture is frozen.

A11. Stage 6 — Decompose the Baseline

Atomize v0.4.6 using the same record process.

Assign each baseline record a disposition: retain, normalize, modify, replace, move, split, merge, retire, archive, or unresolved.

Find inherited orphan rules not mentioned by updates and determine whether they remain canonical, are obsolete, depend on changed systems, or were accidentally unaddressed.

Flag inherited version archaeology, restraint arguments, obsolete terms, placeholders, and tonal contamination.

Gate 6: every baseline requirement has a proposed disposition.

A12. Stage 7 — Bidirectional Migration Ledger

Create one row per atomic record with provenance, type, weight, status, baseline relation, operation, target section, carrier, dependencies, conflict ID, wording fidelity, compilation status, final location, and audit result.

Require bidirectional completeness: every source requirement has a destination or exclusion reason, and every final rule has provenance.

Define EXACT, SEMANTIC, and EDITORIAL wording fidelity.

Split multi-destination records or create explicit document-wide operations.

Gate 7: every source and baseline record has a target or explicit reason for none.

A13. Stage 8 — Executable Change Guide

Generate global terminology and style operations, structural moves and insertions, numbering and cross-reference freeze, section-by-section semantic contracts, appendix routing, and audit instructions.

Each section entry names target, sources, fidelity, STATE contract, guardrails, SHOW, SAY, operations, and source packet.

Keep WHY entries in a separate archive.

Gate 8: the guide is executable and every entry points to full source records.

A14. Stage 9 — SAY Harvest

Scan the whole corpus for compact in-world truths, folk wisdom, animal-scale perceptions, warnings, and recurring principles.

Draft short, memorable, mechanically true SAY candidates free of design jargon.

Validate SAY against STATE, especially universal words such as always, never, every, safe, certain, free, and enough.

Assign canonical placement and related STATE records.

SAY may replace explanatory gloss but never operative rules, causes, exceptions, or guardrails.

A15. Stage 10 — Section Contracts and Packets

Create one contract per final section containing purpose, reader question, opening thesis, definitions, STATE records, guardrails, SHOW, SAY, examples, dependencies, cross-references, exclusions, voice notes, and length range.

Mechanically assemble each packet from the contract, baseline passages, full source passages, rulings, conflicts, terminology, formulas, examples, figures, Change Guide entries, traps, and neighboring summaries.

Do not rely on semantic search alone; use explicit Migration Ledger assignments.

Run packet completeness checks before drafting.

Gate 10: incomplete packets cannot be drafted.

A16. Stage 11 — Compile Sections

Begin from the frozen contract and raise architecture issues rather than redesigning silently.

Write world-facing prose free of rebuttal, version references, rejected alternatives, and model commentary.

Preserve every condition, trigger, exception, tie-breaker, prohibition, unit, and exact formula required by STATE records.

Build SHOW elements explicitly, mark examples as illustrative unless canonical, place only approved SAY, route SILENT material to internal notes, and attach a provenance report.

Gate 11: draft and provenance report are both required.

A17. Stage 12 — Immediate Section Verification

Run mechanical checks for record IDs, formulas, constants, terms, table fields, guardrails, and cross-references.

Use a fresh-context semantic audit with only the contract, packet, draft, and rubric.

Report omissions, weakened conditions, new mechanics, bad examples, lost guardrails, unsupported universals, terminology drift, foil leakage, and unsupported claims.

Repair blockers and substantive findings before integration and update ledger status.

Gate 12: only verified sections enter the integrated draft.

A18. Stage 13 — Assemble the Whole Document

Combine verified sections mechanically while preserving anchors, figure IDs, table IDs, and references.

Run an integration pass for transitions, repeated definitions, openings, local redundancy, and voice without changing mechanics.

Establish one authoritative definition location with compact reminders elsewhere.

Check chapter balance, example distribution, SAY density, prose/table balance, inherited voice, and pasted-spec texture.

Classify repetition as reinforcement, necessary local rule, accidental duplication, or conflicting duplicate.

A19. Stage 14 — Global Acceptance Tests

Run source coverage, baseline coverage, equation fidelity, numerical consistency, guardrail retention, zero foil leakage, definition uniqueness, terminology compliance, SHOW-to-STATE traceability, STATE-to-SHOW sufficiency, SAY truthfulness, example conformance, collision closure, cross-reference resolution, structural integrity, no draft debris, provenance coverage, weight audit, voice consistency, and late-document fidelity tests.

Passing conditions include zero unclassified records, zero records without destination or exclusion reason, no unresolved release blockers, and no unsupported normative paragraphs.

A20. Stage 15 — Independent Whole-Document Audit

Use a fresh thread or model context with the final draft, registry, Migration Ledger, Lexicon, Conflict Ledger, and audit rubric.

Run separate omission/distortion, reader-coherence, and voice/foil-leakage audits.

Require findings tied to source records, final locations, and specific repair actions rather than wholesale rewriting.

A21. Stage 16 — Repair Loop

Classify findings as canon error, omission, contradiction, structural, terminology, presentation, voice, false positive, or optional improvement.

Repair from original records, packets, and rulings rather than auditor summaries alone.

Re-run affected tests, update provenance, and freeze the release candidate.

No post-freeze polish occurs without retesting affected sections and dependencies.

A22. Stage 17 — Release Packaging

Produce canonical Markdown and PDF GDD files.

Create a development changelog outside the GDD body.

Archive WHY entries, rejected options, historical comparisons, supersession reasoning, and deferred briefs.

Publish internal registry, lexicon, migration, conflict, tuning, QA, and build artifacts.

Tag the release and record source hashes, model IDs, prompt versions, parser and script versions, build date, and human approver.

A23. Recommended Operational Sequence

Freeze sources; normalize and validate; build manifest and figure registry; atomize specifications; classify weights and types; reconcile terminology; detect and rule conflicts; design and freeze target architecture; atomize baseline; assign baseline dispositions; build Migration Ledger; generate Change Guide; harvest SAY; create contracts and packets; compile and audit sections; integrate; run global tests and independent audits; repair from sources; freeze, export, tag, and archive.

A24. Relationship to the Original Decision-Log Method

The protocol retains publication weights, STATE/DO/WHY separation, foil quarantine, lantern testing, silent-reversal handling, structural-first ordering, numbering freeze, SAY harvesting, voice greps, and archive separation.

It changes four assumptions: full sources remain available; accountability is atomic; target architecture is independently designed; and compilation and verification are local before global.

A25. Final Release Rule

The edition is complete only when every source claim has a disposition, every final rule has provenance, every conflict has a ruling or explicit unresolved status, and the finished book describes MEDIAN rather than the process by which it was designed.

Appendix B — Recommended Initial Schemas

B1. Source Manifest CSV

source_id,filename,title,version,source_class,scope,status,supersedes,dependencies,authority_notes,processing_order,include_in_compilation,hash

B2. Controlled source classes

baseline
detailed_spec
supplementary_pass
philosophy
research
manifestation
appendix
decision_log
human_ruling

B3. Controlled source statuses

active
provisional
review_required
superseded
archive

B4. Atomic record fields

Short key	Expanded meaning
id	Stable record ID
src	Source ID
loc	Source coordinate
quote	Full operative source passage
claim	Compact normalized claim
type	Content type
weight	STATE, SHOW, SAY, or SILENT
status	Canonical, provisional, example, rationale, historical, tuning, unresolved, superseded, or rejected
owner	System namespace
terms	Term IDs or canonical terms
deps	Dependency record IDs
flags	Review flags

B5. Conflict judgment result

```
{
  "conflict_id": "C-0017",
  "relationship": "direct_conflict",
  "blocking": true,
  "analysis": "...",
  "possible_resolutions": ["..."],
  "recommended_resolution": "...",
  "requires_human_ruling": true
}
```

END OF IMPLEMENTATION SPECIFICATION